

# Cours d'Apprentissage Automatique

## Filière Génie Logiciel

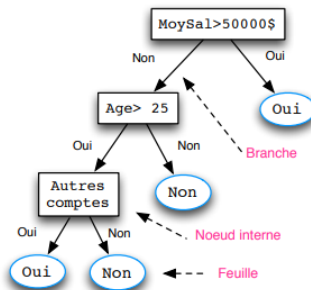
Univerisité Assane Seck de Zigunchor – Département informatique

### **Classification: Arbres de décisions**

17 novembre 2025

# Qu'est-ce qu'un arbre de décision ?

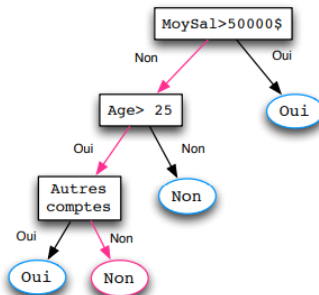
- Modèle d'apprentissage supervisé pour **classer** ou **prédire**.
- Construit par des heuristiques : tests successifs sur les variables.
- Avantage principal : **interprétabilité** — on lit l'arbre comme un ensemble de règles.



**Exemple général** : décider d'*accorder un prêt* selon salaire, âge, autres comptes.

# Exemple

- **Racine** : premier test (ex. MoySal > 50 000\$).
- **Nœuds internes** : tests / conditions.
- **Branches** : issues des tests (Oui / Non ou valeurs multiples).
- **Feuilles** : classe prédite (ex. Prêt = Oui/Non).



# Attributs, règles et traduction en règles

- Les îlots de valeurs d'une variable sont appelés **attributs/instances**.
- Après construction, l'arbre se traduit en un **ensemble de règles** (si ... alors ...).

## Règles issues de l'exemple prêt :

- ① Si MoySal  $>$  50 000  $\Rightarrow$  Prêt = Oui.
- ② Si MoySal  $<$  50 000 et Âge  $<$  25  $\Rightarrow$  Prêt = Non.
- ③ Si MoySal  $<$  50 000 et Âge  $>$  25 et AutresComptes = Oui  $\Rightarrow$  Prêt = Oui.
- ④ Si MoySal  $<$  50 000 et Âge  $>$  25 et AutresComptes = Non  $\Rightarrow$  Prêt = Non.

**Exemple concret (cas) :** Moussa — Age 32, MoySal 45 000,  
AutresComptes = Non.  
 $\Rightarrow$  classé **Prêt = Non**.

## Autre exemple : Jeu de données "Jouer" (14 jours)

| Jour | Ciel    | Temp.   | Humidité | Vent   | Jouer |
|------|---------|---------|----------|--------|-------|
| J1   | Soleil  | Chaude  | Élevée   | Faible | Non   |
| J2   | Soleil  | Chaude  | Élevée   | Fort   | Non   |
| J3   | Couvert | Chaude  | Élevée   | Faible | Oui   |
| J4   | Pluie   | Tiède   | Élevée   | Faible | Oui   |
| J5   | Pluie   | Fraîche | Normale  | Faible | Oui   |
| J6   | Pluie   | Fraîche | Normale  | Fort   | Non   |
| J7   | Couvert | Fraîche | Normale  | Faible | Oui   |
| J8   | Soleil  | Tiède   | Élevée   | Faible | Non   |
| J9   | Soleil  | Fraîche | Normale  | Faible | Oui   |
| J10  | Pluie   | Tiède   | Normale  | Fort   | Oui   |
| J11  | Soleil  | Tiède   | Normale  | Fort   | Oui   |
| J12  | Couvert | Tiède   | Élevée   | Fort   | Oui   |
| J13  | Couvert | Chaude  | Normale  | Faible | Oui   |
| J14  | Pluie   | Tiède   | Élevée   | Fort   | Non   |

On désire élaborer un arbre de décision qui permet de résumer les différentes situations décrites. Plusieurs questions se posent alors, parmi lesquelles :

- Quelle variable discrimine le mieux l'ensemble des données et laquelle doit être considérée en premier ?
- Si plusieurs structures d'arbre répondent aux situations décrites, laquelle doit-on choisir ?

# Algorithme générique de construction d'un arbre de décision

- ① Initialiser l'arbre courant à l'arbre vide ; la racine est le nœud courant
- ② Répéter
  - ① Décider si le nœud courant est terminal (une feuille)
  - ② Si le nœud est terminal Alors
    - Affecter une classe au nœud courant
  - ③ Sinon
    - Sélectionner un test et créer autant de nouveaux fils qu'il y a de possibilités de réponses
  - ④ FinSi
  - ⑤ Passer au nœud suivant
- ③ Jusqu'à l'obtention de l'arbre de décision

# Entropie : notion fondamentale

- L'**entropie** mesure l'incertitude (ou la quantité d'information nécessaire) pour identifier la classe d'un exemple.
- Pour une distribution  $P = (p_1, \dots, p_n)$  :

$$E(P) = - \sum_{i=1}^n p_i \log_2 p_i.$$

- **Propriétés :**
  - $E = 0$  si tous les éléments appartiennent à la même classe (pureté parfaite).
  - $E$  maximal (égale à 1 pour deux classes équilibrées) quand les classes sont équiprobables.
  - Sert à évaluer la *pureté* d'un nœud dans un arbre.

# Entropie et entropie conditionnelle

- Soit un ensemble de données  $T$  composé de  $n$  classes  $(C_1, C_2, \dots, C_n)$ . La distribution des probabilités associée est :

$$P = \left( \frac{|C_1|}{|T|}, \frac{|C_2|}{|T|}, \dots, \frac{|C_n|}{|T|} \right)$$

- L'entropie mesure l'incertitude pour identifier la classe d'un individu :

$$E(T) = - \sum_{i=1}^n \frac{|C_i|}{|T|} \log_2 \left( \frac{|C_i|}{|T|} \right)$$

- Si  $T$  est partitionné selon un attribut  $X$  en sous-ensembles  $(T_1, \dots, T_m)$ , l'entropie conditionnelle est :

$$E(X, T) = \sum_{j=1}^m \frac{|T_j|}{|T|} E(T_j)$$

- **Idée clé** : Une bonne partition réduit l'entropie  $\rightarrow$  meilleure séparation des classes.



## Exemple : entropie de la variable cible (cas 9 vs 5)

On suppose la variable cible “Jouer” avec 9 occurrences “Oui” et 5 occurrences “Non” (total 14).

$$E(\text{Jouer}) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0,940$$

- Interprétation : valeur élevée  $\Rightarrow$  forte hétérogénéité ; beaucoup d'information nécessaire pour prédire correctement.
- Conséquence pour l'arbre : il faut trouver un attribut qui diminue fortement cette entropie.

# Entropie conditionnelle : exemple pour l'attribut *Ciel*

L'attribut *Ciel* a trois valeurs : Soleil, Couvert, Pluie.

| Valeur  | Répartition (Oui, Non) | Entropie $E(T_j)$ |
|---------|------------------------|-------------------|
| Soleil  | (2, 3)                 | 0,971             |
| Couvert | (4, 0)                 | 0                 |
| Pluie   | (3, 2)                 | 0,971             |

L'entropie conditionnelle est :

$$E(\text{Jouer} \mid \text{Ciel}) = \sum_j \frac{|T_j|}{|T|} E(T_j) = \frac{5}{14} \cdot 0,971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0,971 = 0,693.$$

- Remarque : la valeur *Couvert* est pure (entropie 0) : excellente séparation locale.

## Définition

Le **gain d'information** pour un attribut  $X$  (sur l'ensemble  $T$ ) est :

$$\text{Gain}(X, T) = E(T) - E(X, T),$$

où  $E(X, T)$  est l'entropie conditionnelle (moyenne pondérée des entropies des partitions).

- Rôle : mesure la réduction d'incertitude obtenue en scindant selon  $X$ .
- Principe d'ID3 : sélectionner l'attribut qui maximise le *gain* à chaque nœud.
- Un gain élevé signifie : partitions plus homogènes → meilleur pouvoir discriminant.

# Exemples numériques : Gain(Ciel) vs Gain(Humidité)

Données :

$$E(\text{Jouer}) = 0,940.$$

Calculs (valeurs tirées de l'exemple) :

$$E(\text{Jouer} \mid \text{Ciel}) = 0,693 \quad \Rightarrow \quad \text{Gain}(\text{Ciel}) = 0,940 - 0,693 = 0,247.$$

Pour *Humidité* :

$$E(\text{Jouer} \mid \text{Humidité}) = 0,789 \quad \Rightarrow \quad \text{Gain}(\text{Humidité}) = 0,940 - 0,789 = 0,151$$

- Conclusion : **Ciel** réduit davantage l'incertitude ; il est donc choisi en racine par ID3.
- Observation pratique : le gain favorise parfois des attributs à nombreuses modalités (biais possible).

## 1) Rapport de gain (GainRatio) — C4.5

$$\text{GainRatio}(X) = \frac{\text{Gain}(X)}{IV(X)},$$

où  $IV(X)$  est la *intrinsic value* (mesure de la dispersion intrinsèque de  $X$ ).

- But : pénaliser les attributs avec beaucoup de valeurs distinctes (éviter biais).

## 2) Indice de Gini — CART

$$Gini(T) = 1 - \sum_{j=1}^m \left( \frac{|T_j|}{|T|} \right)^2.$$

- Utilisé comme critère de pureté alternatif (CART); on minimise l'indice de Gini.
- $Gini = 0$  pour un nœud pur; plus  $Gini$  est faible, plus la partition est homogène.

# Principe général de l'induction

- Construction **top-down** : on part de la racine et on décline les partitions.
- À chaque nœud : **choisir l'attribut** qui discrimine le mieux les classes selon un critère (entropie/gain, Gini, ...).
- Pseudo-processus :
  - ① Si tous les exemples sont de la même classe  $\Rightarrow$  créer une feuille.
  - ② Sinon, sélectionner le meilleur attribut, partitionner, créer un nœud et répéter récursivement.
  - ③ Arrêt : critère d'arrêt (pureté, pas d'attributs, nombre minimum d'exemples, profondeur max).
- Objectif : maximiser l'homogénéité des feuilles pour obtenir des règles simples et interprétables.

- Conçu par Quinlan (1986) pour des attributs **qualitatifs**.
- Critère : **Gain d'information** basé sur l'entropie.
- Fonctionnement :
  - Calculer le gain pour chaque attribut au nœud courant.
  - Choisir l'attribut de gain maximal.
  - Répéter sur chaque sous-ensemble.
- Limites d'ID3 :
  - Ne traite pas directement les attributs continus (sans discretisation).
  - Pas d'élagage natif ni de gestion avancée des valeurs manquantes.

- Évolution d'ID3 (Quinlan, 1993) : plus robuste et général.
- Apports :
  - Gère les **attributs continus** (seuils automatiques).
  - Traite les **valeurs manquantes**.
  - Intègre un **élagage** (pruning) pour réduire le surapprentissage.
- Critère : **Gain ratio** (rapport de gain) — corrige le biais envers les attributs à nombreuses modalités.
- C5 : implémentation/optimisation plus rapide et mémoire-efficace.



# Algorithme CART

- **CART** = Classification And Regression Trees (Breiman et al.).
- Particularités :
  - Produit des **arbres binaires** (deux branches par nœud).
  - Utilise l'**indice de Gini** comme critère d'impureté.
  - Conçu pour la **classification** et la **régression**.
  - Intègre des méthodes d'**élagage** (par exemple validation croisée).
- Avantage : maniable avec variables numériques et robuste pour la coupe binaire optimale.

# Comparaison et conseils pratiques

- Comparaison synthétique :
  - **ID3** : simple, gain d'information, mieux pour données qualitatives pures.
  - **C4.5** : gain ratio, gère continue & manquantes, élagage.
  - **CART** : Gini, arbres binaires, classification + régression.
- Choix pratique :
  - Données mixtes / continues → privilégier C4.5 (ou CART selon préférence binaire).
  - Besoin d'interprétabilité simple → arbres peu profonds, élagage et post-pruning.
  - Pour meilleure performance → envisager forêts aléatoires (Random Forest) ou boosting.
- Rappel : l'hyperparamétrage (profondeur, min samples leaf...) et l'élagage sont cruciaux pour éviter le surapprentissage.

Algorithme ID3 - Application au jeu de données *Jouer*

# Avantages des arbres de décision

- Utilisés dans de nombreux domaines : finance, médecine, marketing, etc.
- Structures **simples à comprendre** et **faciles à interpréter**.
- Fournissent une **classification rapide et visuelle** via un parcours de l'arbre.
- Chaque chemin représente une **règle explicite et lisible** par l'utilisateur.
- Disponibles dans la plupart des **outils de forage de données** et environnements ML.

- **Sensibles au nombre de classes** : les performances diminuent lorsque le nombre de classes augmente.
- Modèles **peu flexibles** : un changement dans les données nécessite une **reconstruction complète** de l'arbre.
- Risque de **surapprentissage** lorsque l'arbre devient profond ou trop détaillé.
- Solution moderne : **forêts d'arbres de décision** (Random Forest, Breiman 2001) :
  - combinent plusieurs arbres,
  - **réduisent la variance** et améliorent la robustesse,
  - donnent de meilleures performances globales.

# Pourquoi optimiser un arbre de décision ?

- Un arbre non optimisé peut entraîner :
  - **surapprentissage** (arbre trop profond),
  - **sous-apprentissage** (arbre trop simple).
- Objectifs de l'optimisation :
  - améliorer la **généralisation**,
  - réduire la **complexité** du modèle,
  - augmenter la **stabilité**,
  - réduire la sensibilité au **bruit**.
- Optimiser = contrôler la structure de l'arbre et ajuster ses paramètres.

# Contrôle de la croissance : paramètres clés

- Méthodes pour limiter la croissance excessive de l'arbre :
  - **Profondeur maximale** (`max_depth`).
  - **Taille minimale d'une feuille** (`min_samples_leaf`).
  - **Nombre minimum d'exemples pour un split** (`min_samples_split`).
  - **Réduction d'impureté minimale** pour autoriser un split.
- Objectif : éviter la génération de branches trop spécifiques au jeu d'entraînement.

# Élagage (Pruning)

- L'élagage consiste à **supprimer les branches inutiles** pour simplifier l'arbre.
- Deux types :
  - **Pré-élagage (pre-pruning)** :
    - limite la croissance pendant la construction,
    - critères : profondeur max, taille minimale, gain minimum,
    - rapide mais risque de sous-apprentissage.
  - **Post-élagage (post-pruning)** :
    - l'arbre complet est construit puis simplifié,
    - méthodes : réduction d'erreur, coût-complexité (CART),
    - meilleure généralisation, plus coûteux.



# Optimisation de la qualité des splits

- Critères d'impureté adaptés :
  - **Gain Ratio** (C4.5) : moins sensible aux attributs à nombreuses valeurs.
  - **Indice de Gini** (CART) : rapide et efficace.
- Gestion des attributs continus :
  - calcul automatique des **seuils optimaux**.
- Gestion des valeurs manquantes :
  - pondération des branches,
  - imputation simple.

- Pour dépasser les limites des arbres individuels :
  - **Random Forest** :
    - combine plusieurs arbres,
    - réduit la **variance**,
    - améliore la robustesse et les performances.
  - **Gradient Boosting / XGBoost** :
    - arbres ajoutés séquentiellement,
    - chaque nouvel arbre corrige les erreurs des précédents,
    - très performant mais moins interprétable.

# Random Forest : Principe général

- Proposé par **Leo Breiman (2001)** : ensemble d'arbres de décision.
- Idée clé :
  - plutôt qu'un seul arbre instable,
  - construire plusieurs arbres variés,
  - puis **agréger leurs prédictions**.
- Construction :
  - chaque arbre est entraîné sur un **échantillon bootstrap** (tirage avec remise),
  - à chaque split, sélection aléatoire d'un **sous-ensemble de variables**.
- Prédiction finale :
  - classification : **vote majoritaire**,
  - régression : **moyenne** des prédictions.

# Pourquoi le Random Forest fonctionne ?

- Réduit fortement la **variance** par rapport à un arbre unique.
- Les arbres individuels sont :
  - corrélés s'ils utilisent les mêmes variables,
  - **dé-corrélés** grâce :
    - au bootstrap (diversité des données),
    - à la sélection aléatoire de variables (diversité des splits).
- Avantages du modèle :
  - moins de **surapprentissage**,
  - grande **robustesse au bruit**,
  - excellentes performances même sans réglages fins.

# Random Forest : Avantages et limites

- **Avantages :**

- très robuste et précis,
- gère naturellement variables continues et qualitatives,
- tolérant aux valeurs manquantes,
- estime l'**importance des variables**,
- fonctionne bien sur grands volumes de données.

- **Limites :**

- modèle moins **interprétable** qu'un arbre simple,
- peut devenir lourd pour des milliers d'arbres,
- pas idéal pour les situations nécessitant une justification simple de la décision.